Lab Handout 4-A

Dataset Loading, Schema Validation, and 9-Check Audit

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Load the `merged_dataset.csv` produced by Lab Handout 3-H and subject it to the nine mandatory integrity checks that must pass before any preprocessing, feature engineering, or model training is attempted. This handout establishes the *data contract* the rest of Module 4 relies on: if any of the nine checks fail, you stop and fix the data before proceeding.

2 Prerequisites

- Lab Handout 3-H complete (`merged_dataset.csv` and `data_dictionary.md` delivered).
- Python 3.10+ with `pandas`, `numpy`, and `pytest` installed.
- `Code/AIML/` folder created with a `Logs/` subdirectory — this is the home for all Module 4 artifacts.
- `merged_dataset.csv` present in `Code/Tools/CampaignCollect/` (the Module 3 handoff artifact). The script reads from there directly; do not copy it.

3 Background: Why Validate Before Training?

A single corrupted column, an undetected duplicate, or an unconverted two's-complement field will silently propagate into every downstream model. The failure mode is insidious: training runs complete, accuracy numbers look plausible, and the model learns artifacts of the corruption rather than the underlying physics. The nine-check audit here is not optional paperwork — each check has a specific failure mode it prevents.

Sign Sanity on RSSI

The Module 3 collection script (`collect.py`) reads `RegPktRssiValue`, `RegPktSnrValue`, and `RegFeiMsb/Lsb` from the SX1276 and writes them as already-decoded signed decimals into `rssi_dbm`, `snr_db`, and `freq_error_hz`. RSSI should always be non-positive at the receiver; if any `rssi_dbm` value is positive in the merged CSV, something upstream has corrupted the column (e.g., a manual edit, a re-export through Excel, or a regression in `collect.py`). Check 6 enforces this invariant before the values are fed to any model.

4 Procedure

4.1 Step 1 — Set up logging and constants

Create `load_and_validate.py` in `Code/AIML/`. Every Module 4 script writes a timestamped log to `Code/AIML/Logs/` so that troubleshooting (and the final deliverable's audit trail) does not depend on remembering to redirect stdout. Two log files are written per run: a stable `load_and_validate_latest.log` that is overwritten each run for easy tail-following, and a timestamped archive copy that is preserved.

```python
import json
import logging
import sys
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd

SCRIPT_DIR    = Path(__file__).parent
LOG_DIR       = SCRIPT_DIR / "Logs"
MERGED_CSV    = SCRIPT_DIR.parent / "Tools" / "CampaignCollect" / "merged_dataset.csv"
MANIFEST_PATH = SCRIPT_DIR / "validation_manifest.json"

LOG_DIR.mkdir(parents=True, exist_ok=True)

run_stamp     = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
log_archive   = LOG_DIR / f"load_and_validate_{run_stamp}.log"
log_latest    = LOG_DIR / "load_and_validate_latest.log"

log = logging.getLogger("validate")
log.setLevel(logging.DEBUG)
log.handlers.clear()
fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s",
                        datefmt="%Y-%m-%d %H:%M:%S")
for handler in (logging.StreamHandler(sys.stdout),
                logging.FileHandler(log_archive, mode="w", encoding="utf-8"),
                logging.FileHandler(log_latest,  mode="w", encoding="utf-8")):
    handler.setFormatter(fmt)
    log.addHandler(handler)

log.info("=== load_and_validate.py starting ===")
log.info(f"Run stamp: {run_stamp}")
log.info(f"Source CSV: {MERGED_CSV}")
log.info(f"Archive log: {log_archive}")
```

Listing 1: Top of `load_and_validate.py` — imports, paths, logging.

```python
# 23-column schema produced by Module 3's merge_dataset.py (LH3-H).
# Column names match collect.py's CSV writer exactly --- do not rename
# without updating LH3-F (collect.py) and LH3-H (merge_dataset.py).
EXPECTED_COLS = [
    "campaign_id", "run_id", "timestamp_host",
    "rx_timestamp_ms", "tx_timestamp_ms", "inter_arrival_ms",
    "beacon_id", "seq_num", "pkt_type", "mod_kind", "spread_factor",
```

```
    "rssi_dbm", "snr_db", "freq_error_hz", "payload_len",
    "distance_m", "label_sf", "label_mod", "mod_kind_label",
    "label_beacon", "label_pkt", "regime", "split",
]

# Canonical modulation kind mapping, frozen once across Modules 1-5:
#   1 = LoRa, 2 = FSK, 3 = OOK  (Module 1 Table 4; LH1-F; LH3-F/G/H).
VALID_MOD_KINDS = {1, 2, 3}

# Canonical packet types in this dataset: 0=DATA, 1=ACK.
VALID_PKT_TYPES = {0, 1}
PKT_TYPE_DATA, PKT_TYPE_ACK = 0, 1

# Operating regimes: 'clean' = C1, C2, C4 (single-station DATA-only),
# 'contended' = C5 (cooperative-target ping-pong).
VALID_REGIMES = {"clean", "contended"}

# Expected campaign_id family values. C5a and C5b are both stamped as
# "C5" by collect.py (the sub-config is encoded in run_id, not
# campaign_id). C3 was removed from the curriculum in LH3-G.
EXPECTED_CAMPAIGN_IDS = {"C1", "C2", "C4", "C5"}

if not MERGED_CSV.exists():
    log.error(f"merged_dataset.csv not found at {MERGED_CSV}")
    log.error("Run Code/Tools/CampaignCollect/merge_dataset.py first.")
    sys.exit(1)

df = pd.read_csv(MERGED_CSV)
log.info(f"Loaded {len(df):,} rows, {len(df.columns)} columns")
```

Listing 2: Schema constants and dataset load.

4.2 Step 2 — The ten mandatory checks

#	Check	Failure Indicates
1	Schema	Missing or extra column; LH3-H contract violated (must include all 23 columns: `pkt_type`, `regime`, `mod_kind`/`mod_kind_label`, etc.)
2	Row count	Merge was truncated or duplicate rows persist
3	Null-value budget	>1% nulls in any feature column; parser or UART issue
4	Duplicate rows	Same (`run_id`, `beacon_id`, `seq_num`, `pkt_type`) repeated
5	Valid ranges	RSSI, SNR, SF, `mod_kind`, `pkt_type`, or `regime` outside canonical sets
6	RSSI sign sanity	Any `rssi_dbm` value > 0; column was corrupted upstream
7	Label consistency	DATA-row `label_sf` ≠ `spread_factor` or `mod_kind` ≠ `mod_kind_label` (mismatch rate >2%)
8	Class balance	Any class (across all four classification tasks) with <150 samples
9	Split integrity	A `run_id` leaked across train/val/test partitions

Table 1: The nine-check audit. Every check must pass, or the check itself must be explicitly waived with written justification, before continuing to LH4-B.

4.3 Step 3 — Implement the checks

```python
def check_1_schema(df):
    missing = set(EXPECTED_COLS) - set(df.columns)
    extra   = set(df.columns) - set(EXPECTED_COLS)
    assert not missing, f"Missing columns: {missing}"
    if extra:
        print(f"[INFO] Extra columns present (not fatal): {extra}")

def check_2_row_count(df, min_rows=2100):
    # Module 3 expected envelope: C1 (800) + C2 (800) + C4 (up to 800)
    # + C5 (~600) ~= 3000 theoretical, with PER-driven shortfall and
    # C4 may be partial. 2100 is the floor; below that, the merge
    # missed CSVs or the campaign data is too thin for training.
    assert len(df) >= min_rows, \
        f"Row count {len(df)} < expected minimum {min_rows}"

def check_3_nulls(df, budget_frac=0.01):
    frac = df.isna().mean()
    over = frac[frac > budget_frac]
    assert over.empty, f"Columns over null budget: {over.to_dict()}"

def check_4_duplicates(df):
    # Dedup key matches merge_dataset.py: pkt_type is required because
    # Beacon~ID 0x02 produces both DATA (relay) and ACK rows with the
    # same seq_num in C5b.
    key = ["run_id", "beacon_id", "seq_num", "pkt_type"]
    dup = df.duplicated(subset=key).sum()
```

```python
    assert dup == 0, f"{dup} duplicate {tuple(key)} rows"

def check_5_valid_ranges(df):
    assert df["rssi_dbm"].between(-120, 0).all(),  "rssi_dbm out of range"
    assert df["snr_db"].between(-20, 15).all(),    "snr_db out of range"
    # label_sf 7-10 reflects the Module 3 scoping decision (LH3-F):
    # SF11 and SF12 require lora_init() rework that was out of scope.
    assert df["label_sf"].between(7, 10).all(), "label_sf out of range"
    # beacon_id must be Target~A (1), Target~B (2), or Threat (255 = 0xFF)
    assert set(df["beacon_id"].unique()) <= {1, 2, 255}, \
        "beacon_id contains non-canonical values (expected {1, 2, 255})"
    # Canonical mod_kind set (1/2/3). Catches any 0/1 alias drift.
    bad = set(df["mod_kind"].unique()) - VALID_MOD_KINDS
    assert not bad, f"mod_kind contains non-canonical values: {bad}"
    bad = set(df["mod_kind_label"].unique()) - VALID_MOD_KINDS
    assert not bad, f"mod_kind_label contains non-canonical values: {bad}"
    # Canonical pkt_type set (0/1). Any other value indicates a parser bug.
    bad = set(df["pkt_type"].unique()) - VALID_PKT_TYPES
    assert not bad, f"pkt_type contains non-canonical values: {bad}"
    # Regime must be 'clean' (C1, C2, C4) or 'contended' (C5).
    bad = set(df["regime"].unique()) - VALID_REGIMES
    assert not bad, f"regime contains non-canonical values: {bad}"
    # Campaign family check.
    unknown = set(df["campaign_id"].unique()) - EXPECTED_CAMPAIGN_IDS
    if unknown:
        print(f"[INFO] Non-standard campaign_id values: {unknown}")

def check_6_rssi_sign(df):
    # collect.py writes signed rssi_dbm directly from RegPktRssiValue;
    # any positive value indicates upstream corruption (manual edit,
    # Excel re-export, or a regression in the collection pipeline).
    assert df["rssi_dbm"].max() <= 0, \
        f"rssi_dbm contains positive values (max={df['rssi_dbm'].max()}); " \
        "column was corrupted upstream"

def check_7_label_consistency(df, threshold=0.02):
    # ACK rows (pkt_type=1) come from the responding Target whose
    # spread_factor self-report is unaffected by the campaign config's
    # fixed_sf, so we only check DATA rows (Target~A is the originator
    # whose firmware config we control).
    data = df[df["pkt_type"] == PKT_TYPE_DATA]
    if data.empty:
        return
    sf_mismatch  = (data["spread_factor"] != data["label_sf"])
    mod_mismatch = (data["mod_kind"]      != data["mod_kind_label"])
    rate = (sf_mismatch | mod_mismatch).mean()
    assert rate < threshold, \
        f"DATA-row label mismatch rate {rate:.1%} exceeds {threshold:.0%}"

def check_8_class_balance(df, min_per_class=150):
    # Four classification tasks: SF, modulation, Beacon~ID, packet type.
    # Threshold of 150 ensures >=22 test samples per class at 70/15/15.
```

```python
    for col in ["label_sf", "label_mod", "label_beacon", "label_pkt"]:
        counts = df[col].value_counts()
        small = counts[counts < min_per_class]
        if not small.empty:
            print(f"[WARN] {col} classes under {min_per_class}: " \
                  f"{small.to_dict()}")

def check_9_split_integrity(df):
    # Key leakage on run_id, not campaign_id: a single campaign family
    # legitimately contains multiple collection runs, and those may be
    # assigned to different splits. The split must be stable per run_id.
    leak = (df.groupby("run_id")["split"].nunique() > 1)
    leaked = leak[leak].index.tolist()
    assert not leaked, f"Run IDs split across partitions: {leaked}"

for fn in [check_1_schema, check_2_row_count, check_3_nulls,
           check_4_duplicates, check_5_valid_ranges,
           check_6_rssi_sign, check_7_label_consistency,
           check_8_class_balance, check_9_split_integrity]:
    fn(df)
    print(f"[PASS] {fn.__name__}")
```

Listing 3: The nine audit checks (continue in `load_and_validate.py`).

If Check 6 Fails

If `check_6_rssi_sign` fires, do *not* patch around it by negating values in the script. Instead, regenerate `merged_dataset.csv` from the per-run logs by re-running `merge_dataset.py`. If positive values persist after a clean re-merge, inspect the offending `results_*.csv` directly — the corruption originated in `collect.py` (a regression in how it reads `RegPktRssiValue`) or in a manual edit of the per-run file. The data contract is supposed to hand you already-decoded signed values; fixing it at the source guarantees every future load is correct.

4.4 Step 4 — Dataset summary print-out

After the nine checks pass, print a one-screen summary that the rest of Module 4 can paste into reports:

```python
data_mask = df["pkt_type"] == PKT_TYPE_DATA
n_mismatch = int(
    ((df.loc[data_mask, "spread_factor"] != df.loc[data_mask, "label_sf"])
     |
     (df.loc[data_mask, "mod_kind"]      != df.loc[data_mask, "mod_kind_label"]))
    .sum()
)

print("\n=== Dataset Summary ===")
print(f"Total rows:         {len(df):,}")
print(f"Unique campaigns:   {df['campaign_id'].nunique()}")
print(f"Unique runs:        {df['run_id'].nunique()}")
print(f"Regimes:            {df['regime'].value_counts().to_dict()}")
print(f"Splits:             {df['split'].value_counts().to_dict()}")
```

```
print(f"SF classes:          {df['label_sf'].value_counts().sort_index().
    to_dict()}")
print(f"Mod classes:         {df['label_mod'].value_counts().to_dict()}")
print(f"Beacon classes:      {df['label_beacon'].value_counts().sort_index
    ().to_dict()}")
print(f"Pkt-type classes:    {df['label_pkt'].value_counts().sort_index().
    to_dict()}")
print(f"DATA-row mismatches: {n_mismatch}")
```

4.5 Step 5 — Persist a validation manifest

Write the summary to `validation_manifest.json`. LH4-H will read it to confirm that the deployed inference pipeline is consuming the same data contract the models were trained on.

```
import json, datetime
manifest = {
    "generated_at": datetime.datetime.utcnow().isoformat() + "Z",
    "source_file":  "merged_dataset.csv",
    "row_count":    int(len(df)),
    "schema":       list(df.columns),
    "class_counts": {
        "sf":     df["label_sf"].value_counts().sort_index().to_dict(),
        "mod":    df["label_mod"].value_counts().to_dict(),
        "beacon": df["label_beacon"].value_counts().sort_index().to_dict()
            ,
        "pkt":    df["label_pkt"].value_counts().sort_index().to_dict(),
    },
    "regime_counts": df["regime"].value_counts().to_dict(),
    "split_counts":  df["split"].value_counts().to_dict(),
    "data_row_label_mismatches": n_mismatch,
    "checks_passed": 9,
}
with open("validation_manifest.json", "w") as f:
    json.dump(manifest, f, indent=2)
print("Wrote validation_manifest.json")
```

5 Verification Checklist

- ☐ `merged_dataset.csv` loads without errors.
- ☐ All nine audit checks pass (or waivers are explicitly documented).
- ☐ Dataset summary printed and saved.
- ☐ `validation_manifest.json` written to the Module 4 working directory.

Lab Handout 4-A Deliverable

1. `load_and_validate.py`: the loader and 9-check audit, runnable from the command line.
2. `validation_manifest.json`: the machine-readable dataset summary consumed by LH4-H.
3. Audit log: a plain-text or PDF capture of the script's output showing all nine `[PASS]` lines

and the dataset summary.

4. Any waivers: a list of any check that was downgraded from fatal to warning, with a 2–3 sentence written justification per waiver.

Glossary

UART	Universal Asynchronous Receiver/Transmitter	3